



Requisitos del Proyecto Final – .NET 8 MVC Code First, set de requisitos mínimos (obligatorios) y opcionales/extra.

Guía:

1. Configuración inicial

- Proyecto en **.NET 8** con **ASP.NET Core MVC**.
 - Uso de **Entity Framework Core (Code First)** con **migraciones**.
 - Conexión a una **base de datos SQL Server** al menos 6 tablas (o SQLite si corresponde).
 - Configuración de **Inyección de Dependencias (DI)** para servicios.
 - Configuración de herramientas **Front-end** vistas en clases como por ejemplo **sweetalert**.
-

2. Modelo de dominio

- Definir al menos 6 **entidades principales** (ejemplo: Usuario, Producto, Categoría).
 - Las entidades deben tener:
 - **Propiedades obligatorias y opcionales**.
 - **Relaciones** (1 a muchos, muchos a muchos o 1 a 1).
 - **Data Annotations** para validación ([Required], [StringLength], [Range], etc.).
-

3. Capa de datos

- Implementar **DbContext** y configurarlo en `Program.cs`.
 - Crear al menos 4 **migraciones** (ejemplo: inicial + creacion modelos + cambios posteriores).
 - Uso de **Data Seeding** (cargar algunos datos iniciales).
-

4. Funcionalidad en MVC

- CRUD completo para al menos 6 **entidades** (con vistas Razor fuertemente tipadas).
- Uso de **ViewModels** para separar lógica de dominio de la presentación.
- Uso de **Tag Helpers** (asp-for, asp-action, asp-route, etc.).
- Validaciones en cliente y servidor.

- **Dropdowns anidados** con al menos 2 modelos
 - Mostrar un listado de objetos utilizando **stored procedures**
-

5. Experiencia de usuario

- Diseño con **Layout compartido** (`_Layout.cshtml`).
 - Navegación entre secciones con menú.
 - Implementar al menos **una búsqueda o filtrado sensitivo los demás de data table**.
 - Implementar **paginación o clasificación (sorting)** en alguna lista las demás con data table.
-

6. Seguridad y autenticación

- Autenticación con **ASP.NET Core Identity** (registro - inicio de sesión – recuperación-perfiles - etc).
 - **Roles** (ej: Administrador / Usuario administrativo / Cliente).
 - Control de acceso con `[Authorize(Roles = "Administrador")]` `[Area("Admin")]` en controladores o en acciones `if (!await _roleManager.RoleExistsAsync(ROL.Administrador))`.
-

7. Extras opcionales (para indagar)

- API REST complementaria con **Controllers tipo API**.
 - Subida y gestión de archivos (ej: imágenes o PDF).
 - Uso de **Partial Views / View Components** para reutilización.
 - Dockerizar el proyecto.
 - Integración de AdminLTE por ejemplo.
-

8. Entregables

- Código en un repositorio **GitHub/GitLab**, con **commits claros (Ej confirmación de cambios realizados)**.
- Archivo README .md con:
 - Descripción del proyecto.
 - Instrucciones de instalación y ejecución.
 - Capturas de pantalla mostrando las funcionalidades principales (Esperando resolución...).

9. Información del Trabajo:

- El Trabajo podrá realizarse de hasta 2 integrantes pero la nota será individual, **Y** si uno desaprueba lo hacen ambos.

Ejemplo 1: Alumno A = 8 y Alumno B= 6 Resultado: Aprobados

Ejemplo 2: Alumno A = 10 y Alumno B= 4 Resultado: Desaprobados

- Pueden agregar todo lo que crean necesario para su aplicación pero contando con los requisitos pedidos a **excepción** del **punto 7** que es de sugerencias.
- Se acepta uso de **IA** (ChatGPT, DALL·E, Sora, Copilot, Notion AI, etc) para generar código, creación de imágenes, parte del FrontEnd, etc, Sabiendo que les puedo preguntar **que** hace o **como** funciona lo agregado.

10. Mucha Suerte...